

Reviewer Pack — Minimal Index of Quandle Actions Bounds DPLL Path Length

Entry a54636db64fc · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 14:55:05 UTC

Minimal Index of Quandle Actions Bounds DPLL Path Length

Entry ID: a54636db64fc **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-31 14:55:05 UTC

1. Conjecture

Field A (mathematical branch): Quandles and their Actions **Field B** (complexity object): DPLL Search Trees in Complexity Theory

Statement:

The minimal index of quandle actions on the set of all possible assignments to a SAT instance is linearly correlated with its DPLL path length, such that $m_index(\varphi) = \Theta(w_DPLL(\varphi))$, where $m_index(\varphi)$ is the minimal index of quandle action for the satisfiability instance φ and $w_DPLL(\varphi)$ is the DPLL path length.

Rationale (proposer’s reasoning):

Quandles provide a formal structure that can categorize the symmetries in Boolean functions, and their actions could potentially reveal underlying patterns in the search space of SAT problems. A minimal index captures the essential symmetry, which might correlate with the complexity of finding a solution using the DPLL algorithm.

Taxonomy category: QuandleActions (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c79b2a0003cfbbbeb

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between $m_index(\varphi)$ and $w_DPLL(\varphi)$ across all 30 seeds is ≥ 0.8 , with no seed’s $m_index(\varphi) > 10$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 13 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - quandle actions AND DPLL path length - SAT instance AND quandle minimal index - DPLL complexity AND quandle action on assignments

Top relevant hits considered: - [http://arxiv.org/abs/2012.15478v3] N-quandles of links - [http://arxiv.org/abs/0905.3374v3] Symmetric Extensions of Dihedral Quandles and Triple Points of Non-orientable Surfaces - [http://arxiv.org/abs/1808.06276v2] On the knot quandle of the twist-spun trefoil - [http://arxiv.org/abs/1908.01624v1] Learned Clause Minimization in Parallel SAT Solvers - [http://arxiv.org/abs/1505.03340v2] HordeSat: A Massively Parallel Portfolio SAT Solver - [http://arxiv.org/abs/1004.0653v2] Exact Ramsey Theory: Green-Tao numbers and SAT - [http://arxiv.org/abs/1509.07876v3] Complexity Equals Action - [http://arxiv.org/abs/1012.2923v3] Quandle homology and complex volume

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction
from itertools import combinations, product

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    for i in range(rows):
        max_row = i + max(range(i, rows), key=lambda r: abs(matrix[r][i]))
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
        for j in range(cols):
            if i != j:
                factor = Fraction(matrix[j][i], matrix[i][i])
                for k in range(i, cols):
                    matrix[j][k] -= factor * matrix[i][k]
    return matrix

def matrix_multiplication(A, B):
    rows_A, cols_A = len(A), len(A[0])
    rows_B, cols_B = len(B), len(B[0])
    result = [[Fraction(0) for _ in range(cols_B)] for _ in range(rows_A)]
    for i in range(rows_A):
        for j in range(cols_B):
            for k in range(cols_A):
                result[i][j] += A[i][k] * B[k][j]
    return result

def dpll(instance, assignment=None):
```

```

if assignment is None:
    assignment = [-1] * len(instance)

def find_unassigned():
    for i in range(len(assignment)):
        if assignment[i] == -1:
            return i
    return None

unassigned = find_unassigned()
if unassigned is None:
    return True, sum(abs(x) for x in assignment)

literal = 2 * unassigned + (1 if random.choice([True, False]) else 0)
new_assignment = assignment[:]
new_assignment[unassigned] = literal % 2

result, path_length = dpll(instance, new_assignment)
if result:
    return True, path_length

new_assignment[unassigned] = (literal + 1) % 2
result, path_length = dpll(instance, new_assignment)
return result, path_length

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_instance(n):
        clauses = []
        for _ in range(2 ** n):
            clause = [random.choice([-1, 1]) * i for i in range(n)]
            if all(x == 0 for x in clause):
                continue
            clauses.append(clause)
        return clauses

    def quandle_action_group(instance):
        # Placeholder for actual quandle action group computation
        return set()

    n_values = [5, 10, 15, 20, 30, 40]
    metrics = []
    instances_tested = 0
    n_max = 0

    for n in n_values:
        for _ in range(5):
            instance = generate_instance(n)
            assignment = [-1] * len(instance)

            path_length = dpll(instance, assignment)[1]
            m_index = len(quandle_action_group(instance))

            metrics.append({"m_index": m_index, "path_length": path_length})
            instances_tested += 1

```

```

        n_max = max(n_max, n)

    if not metrics:
        return {
            "metric_name": "m_index",
            "metric_value": None,
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    m_indices = [m["m_index"] for m in metrics]
    path_lengths = [m["path_length"] for m in metrics]

    mean_m_index = sum(m_indices) / len(m_indices)
    mean_path_length = sum(path_lengths) / len(path_lengths)

    correlation_coefficient = 0
    if len(metrics) > 1:
        numerator = sum((x - mean_m_index) * (y - mean_path_length) for x, y in zip(m_indices, path_lengths))
        denominator = math.sqrt(sum((x - mean_m_index) ** 2 for x in m_indices)) * math.sqrt(sum((y - mean_path_length) ** 2 for y in path_lengths))
        correlation_coefficient = numerator / denominator

    conjecture_holds = correlation_coefficient >= 0.8 and max(m_indices) <= 10
    counterexample = "" if conjecture_holds else "correlation_coefficient < 0.8 or m_index > 10"

    return {
        "metric_name": "m_index",
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next((seed for seed, result in zip(seeds, results) if not result["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample='{result['counterexample']}' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_718debc6.py", line 137, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_718debc6.py", line 91, in run_trial
    path_length = dpll(instance, assignment)[1]
                  ^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_718debc6.py", line 57, in dpll
    result, path_length = dpll(instance, new_assignment)
                          ^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_718debc6.py", line 57, in dpll
    result, path_length = dpll(instance, new_assignment)
                          ^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_718debc6.py", line 57, in dpll
    result, path_length = dpll(instance, new_assignment)
                          ^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

[Previous line repeated 993 more times]

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_718debc6.py", line 53, in dpll
    literal = 2 * unassigned + (1 if random.choice([True, False]) else 0)
                                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/usr/lib/python3.12/random.py", line 348, in choice
    return seq[self._randbelow(len(seq))]
            ^^^^^^^^^^^^^^^^^^^^^^^^^
```

RecursionError: maximum recursion depth exceeded

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means the Pearson correlation coefficient could not be calculated to verify the conjecture's support condition. | next: Re-run the test without errors to calculate the Pearson correlation coefficient and verify the conjecture's support condition.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	19177
2	propose	ollama_remote	glm4:latest	0	0	26679
3	preregistration	ollama_remote	glm4:latest	0	0	13562
4	novelty	ollama_remote	glm4:latest	0	0	8315
5	novelty	ollama_remote	glm4:latest	0	0	9997
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16171
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	52798
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19536
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20857
10	judge	ollama_remote	glm4:latest	0	0	11060

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 198152 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/a54636db64fc.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a54636db64fc.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/a54636db64fc.tar.gz` (if generated)